Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Matris zinciri çarpımı (matrix chain multiplication) problemi (ekrandan, 00:10:00)
  - Birden fazla matris art arda çarpılacaksa sonuç, çarpımın hangi sırayla yapıldığına bağlı değildir. Ama skaler çarpma sayısı sıraya göre çok değişir. Amaç, en az çarpma gerektiren parantezlemeyi bulmaktır.
  - $p \times q$ ile $q \times r$ boyutlu iki matrisin çarpımı $p \cdot q \cdot r$ çarpma gerektirir. Ardışık matrislerin iç boyutları eşit olmalıdır. A_i matrisi $d_{i-1} \times d_i$ boyutludur.
  - Örnek: A_1 (2×3), A_2 (3×4), A_3 (4×2). $(A_1A_2)A_3$: $2 \cdot 3 \cdot 4 + 2 \cdot 4 \cdot 2 = 24 + 16 = 40$. $A_1(A_2A_3)$: $3 \cdot 4 \cdot 2 + 2 \cdot 3 \cdot 2 = 24 + 12 = 36$. İkinci sıra daha ucuzdur.
  - Üç matriste elle bakmak kolaydır; on ya da yüz matriste dinamik programlama gerekir.
- Matris zinciri için özyinelemeli bağıntı (ekrandan, 00:20:00)
  - $M[i, j]$: $A_i \cdots A_j$ çarpımının en düşük maliyeti. k, parantezin konduğu ayrım noktasıdır.
  - $M[i, j] = 0$ ($i = j$ ise); $M[i, j] = \min_{i \le k < j} \left(M[i, k] + M[k + 1, j] + d_{i-1} \cdot d_k \cdot d_j \right)$ ($i < j$ ise).
  - Bağıntı, üç matrisli örnekteki iki seçeneğin indislerle yazılmasından çıkarıldı: $M[1, 2] + M[3, 3] + d_0 d_2 d_3$ ve $M[1, 1] + M[2, 3] + d_0 d_1 d_3$.
  - Olası parantezleme sayısı Catalan sayılarıyla verilir: $m = n - 1$ için $\frac{(2m)!}{(m+1)!\, m!}$. Dört matriste $\frac{6!}{4!\, 3!} = 5$ seçenek vardır (ekrandan, 00:21:40).
- Tekrar eden alt problemler ve iki tablo (ekrandan, 00:30:00 ve 00:40:00)
  - $M[1, 4]$ için $M[2, 4]$ ve $M[3, 4]$, $M[2, 4]$ için yine $M[3, 4]$ gerekir. Aynı alt problemler tekrar tekrar hesaplanır; memoization şarttır.
  - Maliyetleri tutan M tablosu ve ayrım noktalarını (k) tutan P tablosu kurulur. Köşegen sıfırdır. Tablolar önce $M[1, 2], M[2, 3], M[3, 4]$ gibi komşu çiftlerle, sonra uzunluk artarak çapraz çapraz doldurulur.
- Dört matrisli örnek (ekrandan, 00:40:00)
  - Boyutlar: A_1 3×2, A_2 2×4, A_3 4×2, A_4 2×5; yani $d = (3, 2, 4, 2, 5)$.
  - $M[1, 2] = 24$, $M[2, 3] = 16$, $M[3, 4] = 40$, $M[1, 3] = 28$, $M[2, 4] = 36$, $M[1, 4] = 58$.
  - $P[1, 2] = 1$, $P[1, 3] = 1$, $P[1, 4] = 3$, $P[2, 3] = 2$, $P[2, 4] = 3$, $P[3, 4] = 3$.
  - Okuma: $P[1, 4] = 3$ ise parantez A_3'ten sonra kapanır: $(A_1A_2A_3)A_4$. $P[1, 3] = 1$ ise $A_1(A_2A_3)$. Sonuç: $((A_1)(A_2A_3))A_4$. Önce A_2A_3, sonra A_1 ile, en son A_4 ile çarpılır.
- Karmaşıklık (ekrandan, 00:44:10)
  - Tabloda yaklaşık $\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \approx n^2$ hücre vardır. Her hücre için k en fazla n değer alır. Toplam $O(n^3)$.
- Hash fonksiyonlarında anahtar uzunluğu ve çakışma: önceki dersteki sorunun cevabı (ekrandan, 00:50:25)
  - Anahtar parçalara bölünüp her parça ayrı hash'lenebilir, sonuçlar birleştirilir (toplanır, art arda eklenir vb.).
  - Örnek: 16 bitlik anahtar iki 8 bitlik parçaya bölünür. $h(key_1) = key_1 \bmod 7$, $h(key_2) = key_2 \bmod 11$, $h(key) = h(key_1) \,\|\, h(key_2)$.
  - Kısa anahtarlar 2, 9 ve 16 seçilirse üst parça hep 0 kalır, alt parçalar da mod7'de hep 2 verir. Üç anahtar çakışır. Bütün haneler kullanılsaydı çakışma bu kadar kolay olmazdı.

  - Hoca, evrensel (universal) hash ailelerinde anahtar boyutu ile çakışma riski arasındaki ilişkiyi inceleyen bir makaleyi paylaşacağını söyledi.
- Amortize analiz (amortized analysis): genel fikir (ekrandan, 01:10:00)
  - Şimdiye kadar çoğunlukla en kötü durum incelendi. Ama birbiriyle bağlantılı işlemlerden oluşan bir dizide her işlemin en kötü durumunu toplamak fazla kötümser (pesimistik) bir sınır verir.
  - Amortize analiz, işlem dizisinin tamamına bakarak işlem başına ortalama maliyeti bulur. Üç yöntem anlatıldı: toplam (aggregate) analiz, muhasebe (accounting) yöntemi ve potansiyel (potential) yöntemi.
- Toplam (aggregate) analiz: yığın işlemleri (ekrandan, 01:15:50)
  - PUSH ve POP $O(1)$'dir; n işlem $\Theta(n)$ tutar.
  - `MULTIPOP(S, k)`: yığın boş değilken ve $k > 0$ iken POP yapıp k'yi azaltır. Tek bir MULTIPOP en kötü durumda $O(n)$'dir.
  - Kaba sınır: n işlemin hepsi MULTIPOP olursa $O(n^2)$. Gerçekte ulaşılamaz: boş yığından çekilemez, her çekilen eleman önce PUSH edilmiş olmalıdır. Toplam maliyet $O(n)$, işlem başına ortalama $T(n)/n = O(1)$'dir.
  - İşlem kümesine aynı anda k eleman ekleyen MULTIPUSH eklenirse ortalama $O(1)$ kalmaz; her işlem $O(k)$ olabilir.
- Toplam analiz: ikili sayaç artırma (binary counter increment) (ekrandan, 01:21:30 ve 01:25:10)
  - Sayaç $A[0..k-1]$ dizisidir ($A[0]$ en anlamsız bit). INCREMENT: $i = 0$; $i < A.length$ ve $A[i] = 1$ olduğu sürece $A[i] = 0$, $i = i + 1$; sonra $i < A.length$ ise $A[i] = 1$.
  - Maliyet, değişen (flip) bit sayısıdır. Sıfırdan 16 artırmada toplam maliyetler: 0, 1, 3, 4, 7, 8, 10, 11, 15, 16, 18, 19, 22, 23, 25, 26, 31.
  - $A[i]$, n artırmada $\lfloor n/2^i \rfloor$ kez değişir ($n = 16$: $A[1]$ 8, $A[2]$ 4, $A[3]$ 2 kez). Toplam $\sum_{i=0}^{k-1} \lfloor n/2^i \rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n$. Toplam $O(n)$, işlem başına $O(1)$.
  - İşlem kümesinde azaltma (DECREMENT) da olursa en kötü durum $O(nk)$ olur: $1\,0 \cdots 0$ azaltılınca k bit değişir, sonra artırılınca yine k bit; bu ikisi art arda tekrarlanır.
- Muhasebe (accounting) yöntemi (ekrandan, 01:33:30)
  - Her işleme bir amortize maliyet $\hat{c}_i$ biçilir; gerçek maliyet c_i'dir. Ucuz işlemlere fazladan ücret konur, artan kredi pahalı işlemleri karşılar.
  - Koşul: $\sum_{i=1}^{n} \hat{c}_i \geq \sum_{i=1}^{n} c_i$, yani toplam kredi $\sum \hat{c}_i - \sum c_i$ her an negatif olmamalıdır. Bu sağlanırsa amortize toplam bir üst sınırdır.
  - Yığın: gerçek maliyetler PUSH 1, POP 1, MULTIPOP $\min(s, k)$; amortize maliyetler PUSH 2, POP 0, MULTIPOP 0. Her PUSH bir krediyi elemanın üstünde bırakır; çekilirken bu kredi harcanır. Yığın eleman sayısı negatif olamadığı için kredi de negatif olmaz. n işlem $O(n)$.
  - İkili sayaç: biti 1 yapmanın ve 0 yapmanın gerçek maliyeti 1'dir. Amortize maliyet: 1 yapmak 2, 0 yapmak 0. Her 1 biti üzerinde bir kredi taşır; o bit sıfırlanırken bu kredi kullanılır. Her artırma en fazla bir biti 1 yapar; n artırma $O(n)$.
- Potansiyel (potential) yöntemi (ekrandan, 02:07:15)
  - Fizikteki potansiyel enerjiden esinlenir. D_0 başlangıçtaki veri yapısı, D_i i'inci işlemden sonraki hâlidir. $\Phi(D_i)$ gerçek sayı değerli potansiyel fonksiyonudur.
  - $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$.
  - Toplamda ara terimler birbirini götürür (teleskopik toplam): $\sum_{i=1}^{n} \hat{c}_i = \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0)$.
  - $\Phi(D_0) = 0$ ve her i için $\Phi(D_i) \geq 0$ olacak şekilde tanımlanırsa amortize toplam gerçek toplamın üst sınırıdır.
  - Yığın: Φ = yığındaki eleman sayısı. PUSH: $\Delta\Phi = (s+1) - s = 1$, $\hat{c} = 1 + 1 = 2$. MULTIPOP: $k' = \min(k, s)$, $\Delta\Phi = -k'$, $\hat{c} = k' - k' = 0$. POP: $\hat{c} = 1 - 1 = 0$. Hepsi $O(1)$; n işlem $O(n)$ (ekrandan, 02:14:20).
  - İkili sayaç: Φ = sayaçtaki 1'lerin sayısı. i'inci artırmada t_i bit sıfırlanır ve bir bit 1 olur: $c_i = t_i + 1$, $\Delta\Phi = 1 - t_i$, $\hat{c}_i = 2$. Sayaç sıfırdan başlar, Φ negatif olmaz; n artırma $O(n)$ (ekrandan, 02:20:00).
- Tekrar: geriye doğru yerine koyma (backward substitution) ile yineleme bağıntıları (ekrandan, 02:30:00 ve 02:40:00)

- Yineleme bağıntıları ana teorem (master theorem), yerine koyma, özyineleme ağacı ve tümevarımla çözülebilir.
- $x(n) = x(n-1)+5$, $x(1) = 0$: $x(n) = x(n-i)+5i$; $i = n-1$ ile $x(n) = 5(n-1) = 5n-5$.
- $x(n) = x(n/3)+1$, $x(1) = 1$, $n = 3^k$: $x(3^k) = x(3^{k-i})+i$; $i = k$ ile $x(n) = k+1 = \log_3 n+1$.
- $S(n) = 1^3 + 2^3 + \cdots + n^3$, özyinelemeli algoritma: $n = 1$ ise 1 döndür, değilse $S(n-1)+n\cdot n\cdot n$ döndür. Çarpma sayısı $M(n) = M(n-1)+2$, $M(1) = 0$; çözüm $2n-2$.
- Aynı toplamın döngülü (iteratif) sürümü de $\sum_{i=2}^{n} 2 = 2(n-1)$ çarpma yapar. Karmaşıklık aynıdır ama döngülü sürüm tercih edilir, çünkü her özyinelemeli çağrı parametrelerin yığına atılıp çekilmesi gibi ek maliyet getirir.

Hocanın Özellikle Vurguladığı Kısımlar

- Dinamik programlama önemli bir konudur ve oturması için çok örnek çözmek gerekir.
  - Memoization'ı uygulamayı kolaylaştıran şey, görülen örnek sayısıdır. Matris zinciri bu yüzden ek örnek olarak seçildi.
- Matris zincirinde yalnız en düşük maliyeti vermek yetmez.
  - Kullanıcıya parantezlerin nereye konacağı da söylenmelidir; bu yüzden M tablosunun yanında P (ayrım noktası) tablosu tutulur.
- Dinamik programlamada karmaşıklık = alt problem sayısı × alt problem başına iş.
  - Matris zincirinde $\approx n^2$ hücre, her hücrede $\leq n$ deneme: $O(n^3)$.
- Amortize analizin kilit noktası, maliyetin neye bağlı olduğunu görmektir.
  - Yığında eleman sayısı, ikili sayaçta 1'lerin (flip'lerin) sayısı. Doğru büyüklüğü seçmek problemi çözer.
- Muhasebe ve potansiyel yöntemlerinde koşul ihlal edilmemelidir.
  - Kredi ya da potansiyel hiçbir zaman negatife düşmemelidir; aksi hâlde amortize toplam üst sınır olmaz.
- İşlem kümesi analiz sonucunu değiştirir.
  - MULTIPUSH ya da DECREMENT eklenince işlem başına $O(1)$ sonucu bozulur.

Kısa Tekrar Notları

- $p \times q$ ile $q \times r$ çarpımı: pqr çarpma.
- $M[i,j] = \min_{i\leq k<j}(M[i,k] + M[k+1,j] + d_{i-1}d_k d_j)$, $M[i,i] = 0$.
- $P[i,j]$ en iyi k'yi tutar; parantezleme P'den geriye doğru okunur.
- Parantezleme sayısı Catalan sayısıdır; 4 matriste 5.
- $d = (3,2,4,2,5)$: $M[1,4] = 58$, sonuç $(A_1(A_2A_3))A_4$.
- Matris zinciri DP: $O(n^3)$.
- Amortize analiz: dizinin toplam maliyeti / işlem sayısı.
- Yığın + MULTIPOP: n işlem $O(n)$, işlem başı $O(1)$.
- İkili sayaç: toplam flip $< 2n$, işlem başı $O(1)$; DECREMENT de varsa $O(nk)$.
- Muhasebe: $\sum \hat{c}_i \geq \sum c_i$; PUSH 2, POP 0, MULTIPOP 0; bit 1 yapmak 2, 0 yapmak 0.
- Potansiyel: $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$; $\Phi(D_0) = 0$, $\Phi(D_i) \geq 0$.
- $x(n) = x(n-1)+5$, $x(1) = 0 \to 5n-5$. $x(n) = x(n/3)+1$, $x(1) = 1 \to \log_3 n+1$.
- Küplerin toplamı: $M(n) = M(n-1)+2 \to 2n-2$; döngülü sürüm aynı sınıfta ama daha ucuz.

Detaylı Açıklamalar

- Matris zincirinde bağıntı şöyle kurulur: $A_i \cdots A_j$ çarpımı bir k noktasında ikiye ayrılır. Sol parça $A_i \cdots A_k$ ($d_{i-1} \times d_k$ boyutlu sonuç), sağ parça $A_{k+1} \cdots A_j$ ($d_k \times d_j$ boyutlu sonuç) verir. Toplam maliyet, iki parçanın kendi en iyi maliyetleri ile bu iki sonucu çarpmanın maliyeti $d_{i-1}d_k d_j$'nin toplamıdır. k, i ile $j-1$ arasındaki bütün değerler için denenir ve en küçüğü seçilir. Örneğin $i = 1$, $j = 3$ ise $k = 1$ ve $k = 2$ denenir.

- Dört matrisli örnekte $M[1,4]$ için üç seçenek vardır: $k=1$: $0+36+3\cdot 2\cdot 5=66$; $k=2$: $24+40+3\cdot 4\cdot 5=124$; $k=3$: $28+0+3\cdot 2\cdot 5=58$. En küçüğü 58'dir, $P[1,4]=3$. $M[2,4]$: $k=2$: $0+40+2\cdot 4\cdot 5=80$; $k=3$: $16+0+2\cdot 2\cdot 5=36$. $M[1,3]$: $k=1$: $0+16+3\cdot 2\cdot 2=28$; $k=2$: $24+0+3\cdot 4\cdot 2=48$. Tablonun bütün değerleri python ile doğrulandı. Hoca yalnız $M[1,2]$'yi tahtada açtı, diğerlerini sonuç olarak yazdı.
- Hoca $M[1,4]$'ün $k=1$ seçeneğini anlatırken önce "artı" dedi, sonra d_0, d_1, d_4'ün toplanmadığını, çarpıldığını vurguladı: $A_2A_3A_4$ nasıl çarpılırsa çarpılsın sonuç $d_1\times d_4$ boyutludur ve A_1 ($d_0\times d_1$) ile çarpımı $d_0d_1d_4$ tutar.
- Dinamik programlamanın getirisini görmek için hoca, memoization yapılmasaydı maliyetin ne olacağını düşünmeyi öğrencilere bıraktı. Memoization ile her hücre bir kez hesaplanır, sonraki ihtiyaçta tablodan okunur.
- Bir öğrenci, dinamik programlamanın kaç matristen sonra verimli olduğunu sordu. Hoca bunun matrislerin boyutlarına da bağlı olduğunu, bu konuda bir eşik araştırması yapmadığını, ama çok küçük zincirlerde verimli olmadığını söyledi.
- Önceki dersin hash sorusu: bir öğrenci, daha uzun ve yüksek entropili anahtarın çakışma riskini nasıl azalttığını sormuştu. Hoca, derste kullanılan basit mod tabanlı fonksiyonlarda bunun görülmesinin zor olduğunu kabul etti ve parçalı hash örneğini kurdu. Anahtarın yalnız küçük bir kısmı kullanılınca üst parçanın hash'i hep aynı kalır, alt parça da birkaç değerde çakışır. 2, 9 ve 16'nın mod7 değerleri 2'dir (python ile doğrulandı).
- Toplam analizde de bir en kötü durum vardır, ama tek tek işlemlerin değil, işlem dizisinin en kötü durumu aranır. MULTIPOP'un tek başına $O(n)$ olması, n işlemin $O(n^2)$ tutacağı anlamına gelmez; çünkü çekilebilecek eleman sayısı, yapılmış PUSH sayısını aşamaz.
- İkili sayaç tablosunda $A[0]$ her artırmada, $A[1]$ iki artırmada bir, $A[2]$ dört artırmada bir değişir. 16 artırmada flip sayıları $A[0]$ 16, $A[1]$ 8, $A[2]$ 4, $A[3]$ 2, $A[4]$ 1'dir; toplam 31 (python ile doğrulandı).
- Muhasebe yönteminde hoca restoranlardaki tabak yığınını örnek verdi: her tabak konurken 2 lira ödenir, 1 lirası koyma işine gider, 1 lirası tabağın içinde kalır. Tabak çekilirken bu lira kullanıldığı için POP'a ve MULTIPOP'a ayrıca ücret biçilmez. Bir öğrenci, MULTIPOP'un gerçek maliyeti $\min(s,k)$ iken amortize maliyetinin neden 0 olduğunu sordu. Hoca, MULTIPOP'un birden fazla POP'tan oluştuğunu ve her POP'un kendi tabağındaki krediyle ödendiğini söyledi.
- Potansiyel yönteminde muhasebe yönteminden farkı, krediyi tek tek işlemlere değil veri yapısının bütününe bağlamaktır. Bir öğrenci MULTIPOP'ta potansiyel farkının neden $-k'$ olduğunu sordu. Hoca, önceki durumda $m+k'$ eleman varken k' tanesi çekilince m eleman kaldığını, farkın $m-(m+k')=-k'$ olduğunu açıkladı.
- Ders sonrası not: Tahtadaki potansiyel yöntemi koşulunda (02:07:15) ve hocanın ifadesinde bazen "büyüktür" kullanıldı; doğrusu $\Phi(D_n)\geq\Phi(D_0)$ ve $\sum\hat{c}_i\geq\sum c_i$'dir.
- Ders sonrası not: Hoca ikili sayacın potansiyel analizinin sonunda "total kostu N operasyon için O1" dedi; doğrusu işlem başına $O(1)$, n işlem için toplam $O(n)$'dir. Muhasebe yönteminde de hoca artırmanın amortize maliyetini "bir" diye andı; biçilen maliyetlerle bir artırmanın amortize maliyeti en fazla 2'dir, yani $O(1)$.
- Ders sonrası not: Tahtada iteratif küp toplamının çarpma sayısı $2(n-2+1)=2n$ diye yazıldı (ekrandan, 02:40:00); doğrusu $2(n-1)=2n-2$'dir. Özyinelemeli sürümün sonucuyla aynıdır.
- Yineleme bağıntısı tekrarında bir öğrenci, $x(3^k)$ açılımının ilk satırında üssün yanlış yazıldığını fark etti; hoca satırı $x(3^k)=x(3^{k-1})+1$ olarak düzeltti.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.